

## 4 차 프로젝트

# 결과 보고서

AI 기반 지능형 교통·차량

통합 관제 시스템

차량 관제 · 경로 탐색 · 영상 분석 · 운행 기록

팀원

개발 기간

제출일

---

---

---

## 결과 보고서 목차

### PROJECT REPORT

|                     |    |
|---------------------|----|
| 1. 프로젝트 개요 .....    | 2  |
| 2. 개발 환경 .....      | 4  |
| 3. 시스템 설계 .....     | 6  |
| 4. AI 모델 및 구현 ..... | 11 |
| 5. 성능 평가 .....      | 16 |
| 6. 프로젝트 결과 .....    | 28 |
| 7. 결론 및 향후 계획 ..... | 32 |

본 보고서는 통합 시스템의 구성과 구현 결과, Custom YOLO26-seg 의 학습·평가 자료를 정리한다.

# 1. 프로젝트 개요

## 1.1 프로젝트명

AI 기반 지능형 교통·차량 통합 관제 시스템

## 1.2 프로젝트 목적

본 프로젝트는 차량의 현재 위치와 계획 경로, 카메라 영상의 객체 분석 결과, 과거 운행 기록을 하나의 웹 관제 화면에서 확인하는 것을 목적으로 한다. 차량 제원을 고려한 경로 계산과 가상 배차·도로 통제 시뮬레이션을 제공하여 관제 업무와 상황 대응의 검토를 지원한다.

## 1.3 개발 배경

차량 위치, 운행 경로와 도로 영상이 서로 분리되어 있으면 운영자가 현재 상황을 연결해 판단하기 어렵다. 대형 차량은 통과 가능한 도로의 제약도 고려해야 한다. 이에 지도·경로·영상·기록을 연동하고, 실제 운행에 영향을 주지 않는 가상 환경에서 배차와 도로 통제를 검토하도록 시스템을 구성하였다.

## 1.4 프로젝트 범위

표 1. 프로젝트 기능 영역

| 영역    | 구성 기능                      | 주요 서비스                        |
|-------|----------------------------|-------------------------------|
| 인증·권한 | 로그인, 역할별 API 접근            | Node/Express                  |
| 차량·운행 | 차량 등록, 출발·도착지 설정, 운행 시작·종료 | Node + PostgreSQL/PostGIS     |
| 위치·경로 | BIMS/단말 위치, 차량 제약을 반영한 A*  | Routing/Tracking              |
| 영상·AI | Android 영상, 객체 박스·분할·거리 표시 | Media Relay + Vision          |
| 녹화·재생 | 영상 세그먼트, 탐지·GPS 시간 동기화     | Relay + Node + MinIO + Vision |
| 가상 배차 | 경로 미리보기, 배차 수락, 통제·우회      | Node + Routing                |
| 운영    | HTTPS, TURN, 서비스별 실행 환경    | Nginx + Coturn + Compose      |

본 보고서의 구현 설명은 현재 코드·설정과 설계 문서를 기준으로 한다. 분할 성능은 보관된 학습·평가 기록을 사용한다.

## 1.5 요구사항 분석 및 제약사항

표 2. 주요 요구사항과 구현 구성

| 구분    | 요구사항               | 구현 구성                             |
|-------|--------------------|-----------------------------------|
| 접근 제어 | 역할에 맞는 기능 사용       | JWT 인증, ADMIN/OPERATOR/VIEWER 인가  |
| 차량 관제 | 여러 차량의 위치·계획 경로 조회 | BIMS·단말 GPS 병합, Leaflet 지도        |
| 경로 탐색 | 차량 제약을 반영한 경로 계산   | 높이·중량·너비·접근·회전 제한과 A*             |
| 영상 분석 | 실시간 영상과 객체 결과 조회   | Android·Relay·Vision·Live View 연결 |
| 기록 재생 | 영상·탐지·GPS 동기화 조회   | PTS·epoch·sequence 기반 저장·재생       |
| 가상 배차 | 배차·경유지·도로 통제 시연    | 시나리오별 가상 상태와 경로 버전 관리             |
| 운영    | 서비스 일괄 실행과 보안 점검   | Docker Compose, Nginx TLS, Coturn |

## 1.6 사용자와 권한

표 3. 사용자별 역할

| 사용자        | 역할                       | 관심사            |
|------------|--------------------------|----------------|
| 관리자        | 계정·차량 관리, 권한·설정 관리       | 데이터 적합성과 접근 통제 |
| 관제 운영자     | 차량·경로·영상 관제, 운행·가상 배차    | 상황 파악과 대응      |
| 조회자        | 지도·영상·운행 기록 조회           | 읽기 전용 정보 확인    |
| Android 단말 | 배정 운행 확인, 카메라·GPS/IMU 송출 | 운행과 송출의 안정성    |

## 1.7 제약사항

지역 범위는 부산 도로망이며, 차량 프로파일은 시연용이다. Live View 는 단일 Android 스트림을 다룬다. GPU 추론·실제 Android 송출은 원래 장비에서 검증해야 한다. 실제 차량의 운행 중 경로 재배포, SUMO 와 Tauri 는 현재 통합 범위에서 제외된다.

## 2. 개발 환경

### 2.1 개발 언어 및 기술 스택

표 4. 서비스별 개발 기술

| 영역      | 기술                                | 용도                 |
|---------|-----------------------------------|--------------------|
| 관제 웹    | JavaScript, HTML/CSS, Leaflet     | 지도·워크스페이스·영상 연동    |
| 업무 백엔드  | TypeScript, Express 5, Prisma 7   | API, 인증, 업무 데이터 관리 |
| 경로·추적   | Python, FastAPI, OSM 도로 그래프       | 차량 제한 판정·A*·위치 병합  |
| 미디어 릴레이 | Go 1.22, Pion WebRTC v4           | H.264 수신·전달·녹화     |
| 비전      | Python 3.12, PyTorch, Ultralytics | 분할·추적·거리 추정        |
| 차량 단말   | Kotlin, CameraX, WebRTC, OkHttp   | 카메라·GPS/IMU·운행 제어  |
| 업무 저장소  | PostgreSQL 17, PostGIS 3.5        | 업무·공간 데이터          |
| 객체 저장소  | MinIO S3 API                      | 녹화 영상 세그먼트         |
| 인프라     | Docker Compose, Nginx, Coturn     | 컨테이너 실행·HTTPS·TURN |

### 2.2 주요 라이브러리

Node 백엔드는 Express 5, Prisma 7 과 PostgreSQL 어댑터를 사용한다. Zod 로 요청을 검증하고 jose·Argon2 로 토큰과 비밀번호를 처리하며 Pino 로 로그를 기록한다. 관제 웹은 별도 프레임워크 빌드 과정 없이 JavaScript 와 Leaflet 로 구성된다.

Vision 은 Python 3.12, FastAPI, PyTorch, 커스텀 Ultralytics 와 영상 처리 라이브러리를 사용한다. 현재 잠금 파일에는 PyTorch 2.12.1+cu130, torchvision 0.27.1+cu130 이 기록되어 있다. 서비스의 패키지 설정은 모델 학습 환경과 구분하여 관리한다.

### 2.3 개발 도구

서비스별로 npm, uv, Go module 과 Gradle 을 사용한다. Git 으로 변경을 관리하고 OpenAPI 로 업무 API 를 문서화한다. 테스트 구성에는 Vitest·Supertest, Python 테스트, Go 테스트와 Android 단위 테스트가 포함되어 있다.

근거: node/package.json, 각 서비스 pyproject.toml·uv.lock, Go module, Android Gradle 설정. 표의 버전은 저장소 설정 기준이다.

## 2.4 데이터 저장소 및 운영 환경

표 5. 데이터 저장 구조

| 저장소                | 저장 데이터                    | 담당·용도                 |
|--------------------|---------------------------|-----------------------|
| PostgreSQL/PostGIS | 계정, 차량, 운행, 경로, 위치, 가상 상태 | Node 의 업무 데이터 및 공간 쿼리 |
| MinIO              | 녹화 영상 세그먼트 객체             | Relay 업로드, 브라우저 재생    |
| Routing 메모리        | 도로 그래프, 위치 스냅샷, 계산 캐시     | 경로 계산과 일시적 위치 추적      |
| routing_state      | 선택한 BIMS 실시간·재생 소스와 보정 설정 | 소스 설정의 재시작 후 유지       |
| Relay 임시 저장        | 녹화·업로드 저널과 미디어 버퍼         | 영상 처리·업로드 상태 관리       |

## 2.5 운영 및 배포 환경

표 6. 주요 서비스의 외부 접점

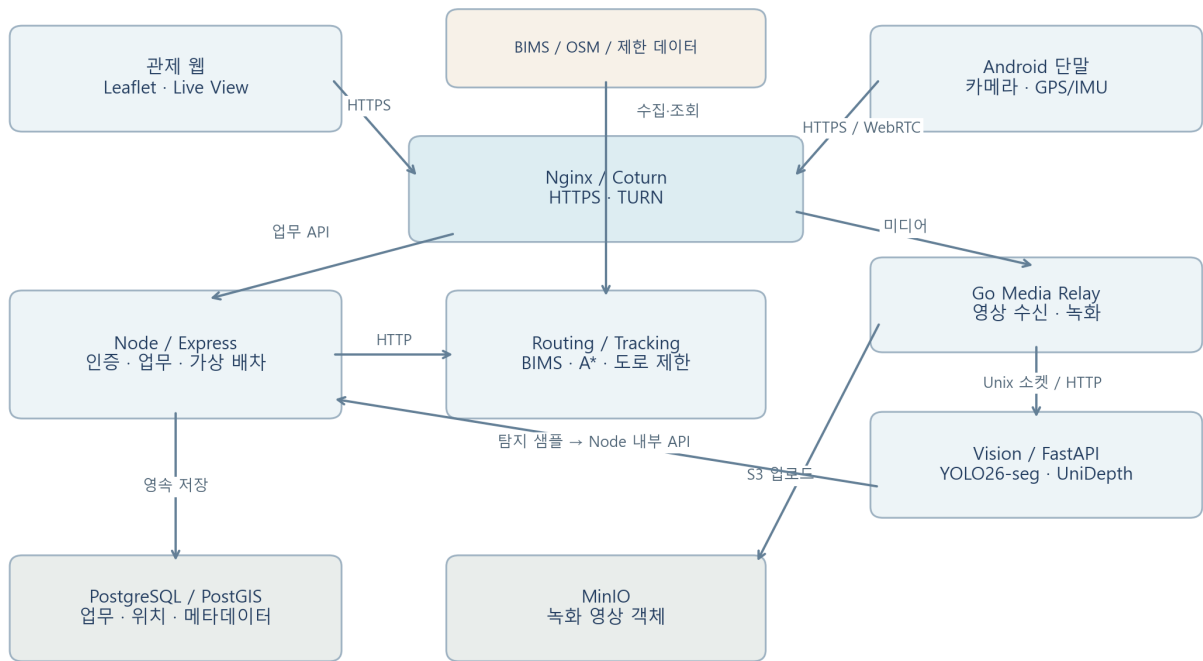
| 서비스                | 접점                                 | 외부 노출   |
|--------------------|------------------------------------|---------|
| Nginx              | 39001 관제 / 39002 영상·단말 / 39003 저장소 | HTTPS   |
| Coturn             | 39004~39007                        | TURN    |
| Node / Routing     | 3000 / 8000                        | 내부 네트워크 |
| Vision / Relay     | 39011 / 39012                      | 내부 네트워크 |
| PostgreSQL / MinIO | 5432 / 9000                        | 내부 네트워크 |

운영·개발 Compose 설정을 구분한다. 운영용 Vision 코드는 이미지에 포함되고 모델 디렉터리는 읽기 전용 디렉터리 바인드 마운트로 연결된다. Compose 의 기본 분할 모델 경로는 a4\_best.engine 이며, Python 단독 실행 설정의 기본 경로는 yolo26s-seg.pt 이다.

관제 AI 도우미는 별도 LLM 서비스와 외부 검색 저장소를 사용하는 부가 기능이다. 핵심 영상 모델 설명은 Custom YOLO26-seg 와 UniDepth 를 중심으로 정리한다.

### 3. 시스템 설계

#### 3.1 전체 시스템 구성



영상·추론과 업무 처리를 서비스 경계에서 연결

그림 1. 통합 시스템 서비스 구성

표 7. 계층별 구성과 책임

| 계층     | 구성요소                         | 책임                        |
|--------|------------------------------|---------------------------|
| 클라이언트  | 관제 웹 / Android               | 지도·영상 표시 / 카메라·센서 송출      |
| 외부 접점  | Nginx / Coturn               | TLS 종단, 경로 라우팅, NAT 통과    |
| 업무 처리  | Node / Express               | 인증·권한, 업무 API, 가상 시뮬레이션   |
| 계산     | Routing / Tracking           | 도로 그래프, A*, BIMS·단말 위치 병합 |
| 미디어·비전 | Go Relay / FastAPI Vision    | 영상 수신·녹화, 객체 분할·거리 추정     |
| 저장     | PostgreSQL / PostGIS / MinIO | 업무·공간 데이터, 영상 객체          |

Node 가 업무·영속 데이터를 관리하고, Routing 은 그래프와 일시적 추적 상태를 관리한다. 미디어 처리와 AI 추론은 Node 를 경유하는 고빈도 영상 전달 방식으로 합치지 않고 별도 서비스에서 수행한다.

추가 제공된 시스템 설계 문서의 서비스 경계를 적용하되, 전달 프로토콜과 부가 서비스는 현재 코드·Compose 설정에 맞춰 정리하였다.

### 3.2 서브시스템과 통신 인터페이스

표 8. 주요 서브시스템

| 서브시스템            | 핵심 책임                        | 인터페이스                                   |
|------------------|------------------------------|-----------------------------------------|
| Node/Express     | 인증·차량·운행·기록·배차, API 파사드      | /api/v1/*, /internal/*                  |
| Routing/Tracking | BIMS, 단말 위치 병합, A*, 통제 간선 계산 | /internal/vehicles, /internal/routing/* |
| Media Relay      | WebRTC 수신, 세션 검증, 녹화·GPS 전달  | /offer/android, Unix 소켓, S3             |
| Vision           | 객체 분할·추적·거리, 프레임·센서 매칭       | 추론 feed, /ws/playback, 내부 API           |
| 관제 웹             | 지도·차량·경로·영상·재생·가상 배차         | HTTPS REST, iframe, postMessage         |
| Android          | 배정 운행 제어, 카메라·GPS/IMU 송출     | HTTPS REST, WebRTC DataChannel          |

### 3.3 주요 데이터 흐름

표 9. 서비스 간 데이터 전달

| 출발 → 도착               | 전달 방식                     | 내용                       |
|-----------------------|---------------------------|--------------------------|
| 관제 웹 → Node           | HTTPS REST + JWT          | 인증·조회·운행·배차 요청           |
| Node → Routing        | 내부 HTTP                   | 위치 스냅샷·경로·통제 계산          |
| Android → Relay       | WebRTC / DataChannel      | H.264, GPS/IMU, QR 이벤트   |
| Relay → Vision        | Unix 소켓 / 내부 HTTP         | H.264 feed 와 프레임 제어 / 센서 |
| Relay → Node / MinIO  | 내부 HTTP / S3              | 세션 검증·GPS·세그먼트 / 영상      |
| Vision → Node         | 내부 HTTP                   | 녹화 탐지 샘플                 |
| 브라우저 ↔ Vision / MinIO | WebSocket / presigned GET | Live View 데이터 / 녹화 영상    |

현재 Relay 의 yolofeed 는 Unix 소켓으로 Vision 과 연결된다. 설계 문서의 미디어·비전 서비스 역할을 유지하면서 실제 전달 경로를 구조도와 인터페이스 표에 반영하였다.

### 3.4 데이터베이스 설계

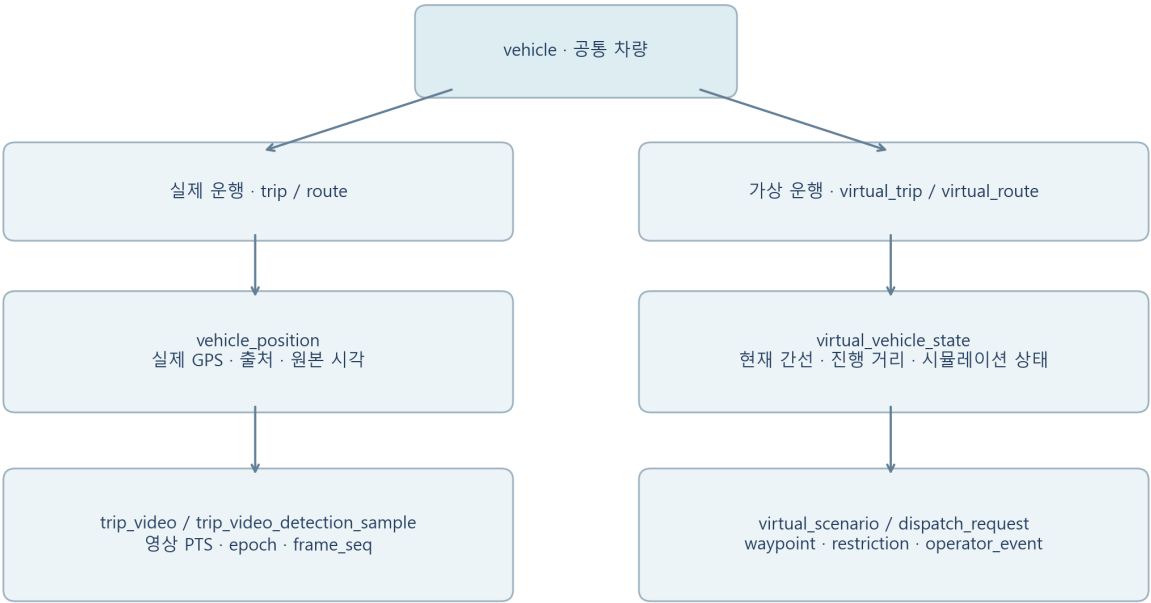


그림 2. 실제 운행과 가상 배차의 주요 관계

표 10. 핵심 데이터 그룹

| 그룹    | 주요 테이블                                                                  | 내용                |
|-------|-------------------------------------------------------------------------|-------------------|
| 계정·차량 | platform_account, vehicle, driver                                       | 사용자 역할·차량 출처·제원   |
| 운행·경로 | trip, route, vehicle_position                                           | 운행 상태·버전 경로·GPS   |
| 영상 기록 | trip_video, trip_video_detection_sample                                 | 영상 객체·PTS·탐지 JSON |
| 가상 운영 | virtual_scenario, virtual_dispatch_request                              | 시나리오·배차 요청·수락 상태  |
| 가상 주행 | virtual_trip, virtual_route, virtual_vehicle_state                      | 경로 버전·현재 간선·이동 상태 |
| 가상 통제 | virtual_trip_waypoint, virtual_road_restriction, virtual_operator_event | 경유지·통제·감사 기록      |

실제 위치와 가상 위치는 별도 테이블로 관리한다. 공간 데이터는 PostGIS geography 를 사용하고, 관련 읽기·쓰기는 TypedSQL 등 공간 SQL 로 처리한다. 상태값은 문자열과 데이터베이스 CHECK 제약으로 관리한다.

### 3.5 데이터 정합성 및 화면 설계

#### 3.5.1 데이터 정합성

표 11. 주요 식별·정합성 기준

| 항목          | 기준                                                    | 목적             |
|-------------|-------------------------------------------------------|----------------|
| 차량·경로·위치 출처 | vehicle_source / route_source / telemetry_source      | 출처 혼동 방지       |
| 단말 GPS      | recording_session_id + source_timestamp_ns            | 같은 GPS 의 맥등 저장 |
| 영상·탐지       | recordingSession + relay epoch + frame sequence + PTS | 원본 영상과 결과 연결   |
| 가상 배차       | idempotency_key 와 트랜잭션                                | 중복 수락·운행 생성 방지 |
| 가상 경로·명령    | routeVersion / commandVersion / sequence              | 오래된 갱신 적용 방지   |

object\_class, detection\_event, event\_image, route\_deviation, alert 와 transport\_goal 은 확장을 위한 스키마에 포함되어 있다. 현재 녹화 탐지 결과의 저장 경로는 trip\_video\_detection\_sample 이므로, 스키마 정의와 운영 기능을 구분한다.

#### 3.5.2 관제 화면

표 12. 워크스페이스별 주요 화면

| 화면        | 표시·조작 내용                          |
|-----------|-----------------------------------|
| 일반 관제     | 차량 지도, 선택 차량, 운행·계획 경로, Live View |
| 운행 기록     | 녹화 목록, 영상 타임라인, 탐지·GPS 재생         |
| 가상 배차     | 차량·출발지·도착지·경유지, 경로 미리보기, 배차 요청    |
| 도로 통제     | 정체·차단 구역, 영향 도로, 차량별 자동 추종        |
| 관제 AI 도우미 | 현재 차량 상태를 포함한 질의와 근거 기반 답변        |

GPS 지연 상태와 데이터 소스는 화면에 구분하여 표시한다. 기록 경로를 이용한 위치 보정은 운영자가 활성화한 경우에 적용한다. 화면에 표시되는 추정·재생 위치를 실제 수신 GPS 와 혼동하지 않도록 출처를 유지한다.

### 3.6 영상·센서 동기화와 가상 주행

#### 3.6.1 영상·GPS/IMU 처리

- 1. Android 가 배정 운행과 녹화 세션으로 WebRTC 연결을 요청한다.
- 2. Relay 가 Node 에서 세션의 trip-vehicle 을 검증하고 영상과 센서를 수신한다.
- 3. GPS 는 Node 에 저장하며 GPS/IMU 는 Vision 으로 전달한다.
- 4. Vision 이 원본 프레임 시각과 센서를 매칭하고 탐지 결과를 생성한다.
- 5. 녹화 영상과 탐지 샘플을 같은 PTS-epoch-sequence 기준으로 재생한다.

#### 3.6.2 가상 배차·도로 통제

표 13. 가상 운영 주요 동작

| 상황          | 처리                                    |
|-------------|---------------------------------------|
| 배차 수락       | 요청 수락 후 가상 운행·경로·상태를 생성하고 주행 시작       |
| 자동 추종 ON    | 통제 변화 시 남은 경로를 다시 계산해 적용              |
| 자동 추종 OFF   | 정체에서는 기존 주행 유지, 차단 앞에서는 운영자 대응 대기     |
| OFF → ON    | 현재 위치와 남은 경유지·목적지를 기준으로 즉시 재계산        |
| 점유 도로 차단 요청 | 가상 차량이 점유한 도로를 포함하면 409 ROAD_OCCUPIED |
| 경로 없음       | NO_ROUTE 등 상태로 표시하고 운영자 대응 지원         |

가상 시뮬레이션은 브라우저 연결과 분리된 서버 상태로 진행한다. 실제 운행 영상·녹화와 가상 차량의 이동을 함께 사용하는 데이터 경로로 합치지 않는다.

## 4. AI 모델 및 구현

### 4.1 주요 AI 모델

표 14. 주요 모델과 역할

| 모델                | 입력     | 역할·결과              |
|-------------------|--------|--------------------|
| Custom YOLO26-seg | 영상 프레임 | 객체 클래스·박스·인스턴스 마스크 |
| UniDepth          | 영상 프레임 | 거리 측정              |

### 4.2 Custom YOLO26-seg 실험 구성

분할 모델은 YOLO26s-seg 를 기준으로 업샘플링, 문맥 특징과 마스크 디코더를 변경하였다. 구조별 구현을 A0~A4 로 구분하고, 저장된 학습 기록을 사용하여 성능을 비교한다.

표 15. 분할 모델 구조별 구성

| 실험 | 구조                                | 변경 내용                             |
|----|-----------------------------------|-----------------------------------|
| A0 | YOLO26s-seg 기본                    | 기본 백본·Neck·분할 헤드                  |
| A1 | CARAFE                            | Top-down 업샘플링에 내용 기반 특징 재조합       |
| A2 | ASPP                              | 고수준 특징에 병렬 팽창 합성곱 문맥 결합           |
| A3 | CARAFE + ASPP                     | 내용 기반 업샘플링과 다중 문맥 결합              |
| A4 | CARAFE + ASPP + DeepLabV3+ 방식 디코더 | P2 의 상세 특징과 고수준 특징을 결합하여 Proto 생성 |

### 4.3 모델 변경 목적

CARAFE 는 업샘플링 과정에서 입력 특징의 내용을 반영한다. ASPP 는 서로 다른 수용 영역의 문맥을 결합한다. A4 의 디코더는 낮은 stride 의 상세 특징을 마스크 생성에 사용하여 객체 경계를 보완하도록 구성하였다. 각 변경의 실제 효과는 성능 평가에서 확인한다.

4.4 A4 분할 모델 구조

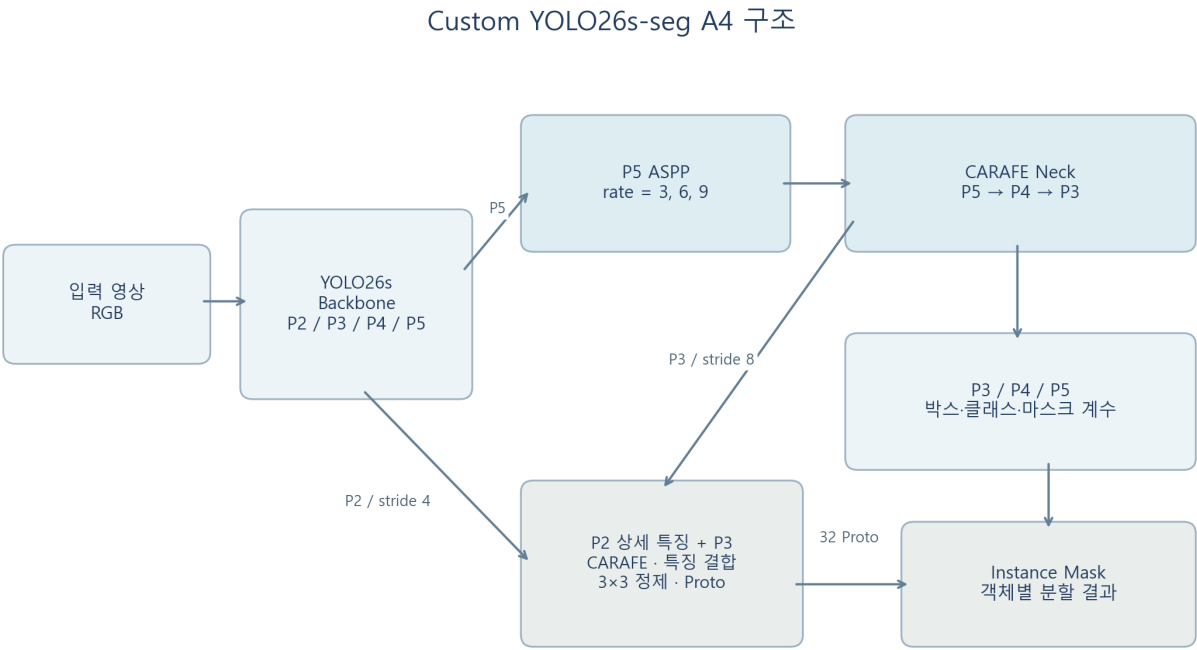


그림 3. CARAFE-ASPP-DeepLabV3+ 방식 디코더를 결합한 A4

표 16. A4 주요 모듈 설정

| 모듈                   | 설정                                      | 역할                        |
|----------------------|-----------------------------------------|---------------------------|
| ASPP                 | P5 / stride 32, dilation 3-6-9          | 병렬 1×1·팽창 3×3·전역 풀링 문맥 결합 |
| Neck CARAFE          | scale 2, 재조합 kernel 5, encoder kernel 3 | P5→P4 와 P4→P3 업샘플링        |
| Low-level projection | P2 / stride 4, 1×1 conv, 48 채널          | 상세 특징의 채널 축소              |
| Decoder              | P3 특징 CARAFE 업샘플링, concat, 3×3 conv 두 번 | P2 와 문맥 특징 정제             |
| Proto 출력             | 32 개 마스크 prototype                      | 객체별 계수와 결합할 분할 표현         |
| 검출 경로                | P3·P4·P5 의 기존 검출 구조 유지                  | 클래스·박스·마스크 계수 생성          |

학습 시 보조 semantic 분기가 사용된다. 최종 객체별 마스크는 Proto 와 객체별 마스크 계수로 구성되며, 디코더의 결합 대상은 P2 상세 특징과 Neck 에서 얻은 P3 특징이다.

근거: experiments/configs/yolo26-seg-carafe-aspp-deeplabv3plus.yaml 및 ultralytics/nn/modules/custom\_seg.py.

## 4.5 탐지 클래스와 학습 설정

학습은 COCO 2017 의 80 개 클래스를 기준으로 수행하였다. 관제에서 주로 확인하는 교통 관련 클래스는 다음 여섯 종류이며, 화면의 클래스 선택은 체크포인트의 클래스 이름을 기준으로 처리한다.

표 17. 주요 교통 객체 클래스

| COCO ID | 클래스        | 표시명  |
|---------|------------|------|
| 0       | person     | 사람   |
| 1       | bicycle    | 자전거  |
| 2       | car        | 승용차  |
| 3       | motorcycle | 오토바이 |
| 5       | bus        | 버스   |
| 7       | truck      | 트럭   |

표 18. A0~A4 공통 학습 설정

| 항목                  | 설정값                               |
|---------------------|-----------------------------------|
| 데이터 설정              | coco.yaml / COCO 2017             |
| 입력 크기               | 640 × 640                         |
| 학습 epoch / patience | 100 / 100                         |
| 배치 크기               | 32                                |
| Optimizer 설정        | Ultralytics auto                  |
| 학습률 설정              | lr0 = 0.01, lrf = 0.01            |
| Seed / 데이터 비율       | 0 / 1.0                           |
| 정밀도                 | AMP 활성화                           |
| 학습 방식               | 실험별 단일 GPU 실행                     |
| 기록                  | args.yaml, results.csv, 가중치 및 그래프 |

설정은 각 실행 폴더의 args.yaml 을 기준으로 정리하였다. 학습 장비 설명과 서비스 실행 의존성은 별도 환경으로 구분한다.

## 4.6 실시간 추론 및 관제 연동

Vision 서비스는 분할 모델인지 확인한 뒤 추론을 시작한다. 클래스 설정은 체크포인트의 클래스 맵으로 해석하며, 추적 모드에서는 ByteTrack 설정과 유지 상태를 사용한다. 프레임의 박스·클래스·confidence·마스크와 추적 ID 를 관제용 결과로 정리한다.

표 19. 추론·표시 설정

| 항목               | 현재 구성                                                   |
|------------------|---------------------------------------------------------|
| 모델 형식            | 분할 체크포인트 또는 TensorRT engine                             |
| Compose 기본 가중치   | a4_best.engine                                          |
| Python 설정 기본 가중치 | yolo26s-seg.pt                                          |
| 입력 크기            | auto 또는 지정 크기, 최대 크기 설정                                 |
| 정밀도              | GPU half 설정 또는 모델에 맞는 정밀도                               |
| 결과               | class, confidence, bbox, mask, track_id, distance_m, 상태 |
| 표시 방식            | 클래스별 박스 / 분할 선택                                         |
| 녹화 결과 저장         | Node 내부 API → trip_video_detection_sample               |

### 4.6.1 동기화 및 실패 상태

결과에는 원본 프레임 식별 정보가 포함되며 프레임과 탐지의 연결은 도착 순서만으로 처리하지 않는다. 거리 정보가 없는 경우 숫자를 임의로 채우지 않고 상태와 함께 전달한다. 녹화 저장 경로는 추론과 분리된 큐로 처리한다.

### 4.6.2 관제 AI 도우미

부가 기능으로 현재 차량 스냅샷과 운송 관련 문서 근거를 활용하는 관제 AI 도우미가 구성되어 있다. Node 가 별도 LLM 서비스에 요청하고, 검색 문맥을 활용한 한국어 답변을 관제 패넬에 표시한다.

서비스 구성은 코드·설정 확인 결과이며, 본 문서 작성 중 실제 GPU 추론이나 Android 송출을 수행한 결과는 아니다.

## 4.7 UniDepth 거리 측정 모델

UniDepth 는 영상 기반 거리 측정을 담당한다. 상세 모델 구성과 학습·실행 자료를 추가할 수 있도록 아래 표를 마련하였다.

표 20. UniDepth 모델 구성·학습 설정

| 항목              | 내용 |
|-----------------|----|
| 모델명 / 버전        |    |
| 모델 구조           |    |
| 입력 / 출력         |    |
| 데이터셋            |    |
| 학습·검증 분할        |    |
| 입력 해상도          |    |
| 학습 epoch        |    |
| 배치 크기           |    |
| Optimizer / 학습률 |    |
| 가중치 / 실행 환경     |    |

### UniDepth 모델 구조도·연동 흐름 작성 공간

## 5. 성능 평가

### 5.1 평가 지표와 기록 기준

표 21. 분할·탐지 평가 지표

| 지표            | 의미                          |
|---------------|-----------------------------|
| Precision     | 모델의 예측 중 정답으로 판단된 비율        |
| Recall        | 정답 객체 중 모델이 찾아낸 비율          |
| Box mAP50     | 박스 IoU 0.50 기준의 클래스별 AP 평균  |
| Box mAP50-95  | 박스 IoU 0.50~0.95 기준의 AP 평균  |
| Mask mAP50    | 객체 마스크 IoU 0.50 기준의 AP 평균   |
| Mask mAP50-95 | 마스크 IoU 0.50~0.95 기준의 AP 평균 |
| 분할 손실         | 학습·검증에서 계산한 분할 오차 항목        |

### 5.2 평가 자료 구분

첫 번째 비교는 저장된 A0~A4 학습 검증 기록이다. 실행별 Mask mAP50-95 가 가장 높은 epoch 를 선택하고, 해당 행의 모든 지표를 함께 사용한다. 표의 최고 epoch 는 이 선택 기준을 뜻하며, 가중치 파일의 별도 선정 기준과 구분한다.

두 번째 비교는 기존 모델 비교 문서에 기록된 공통 조건 검증이다. A0, A4 와 공식 체크포인트를 같은 COCO val2017 조건에서 평가한 자료를 정리하였다. 학습 검증 기록과 공통 조건 결과는 서로 다른 평가 기록이다.

데이터 출처: 각 실행 폴더의 results.csv 및 experiments/notes/yolo26\_seg\_model\_comparison.md. 새 학습·검증은 실행하지 않았다.

## 5.3 A0~A4 학습 기록 비교

표 22. 실행별 최고 epoch 의 Mask 지표

| 실험 | Epoch | Precision | Recall  | mAP50   | mAP50-95 |
|----|-------|-----------|---------|---------|----------|
| A0 | 91    | 0.71449   | 0.55642 | 0.60960 | 0.39474  |
| A1 | 92    | 0.71518   | 0.56126 | 0.60992 | 0.39619  |
| A2 | 91    | 0.70948   | 0.55764 | 0.61014 | 0.39533  |
| A3 | 91    | 0.70948   | 0.55764 | 0.61014 | 0.39533  |
| A4 | 92    | 0.72381   | 0.56082 | 0.61404 | 0.39683  |

표 23. 동일 epoch 의 Box 지표

| 실험 | Epoch | Precision | Recall  | mAP50   | mAP50-95 |
|----|-------|-----------|---------|---------|----------|
| A0 | 91    | 0.71405   | 0.57778 | 0.63649 | 0.46617  |
| A1 | 92    | 0.72561   | 0.57583 | 0.63665 | 0.46789  |
| A2 | 91    | 0.70833   | 0.57733 | 0.63635 | 0.46715  |
| A3 | 91    | 0.70833   | 0.57733 | 0.63635 | 0.46715  |
| A4 | 92    | 0.72519   | 0.58144 | 0.64082 | 0.47056  |

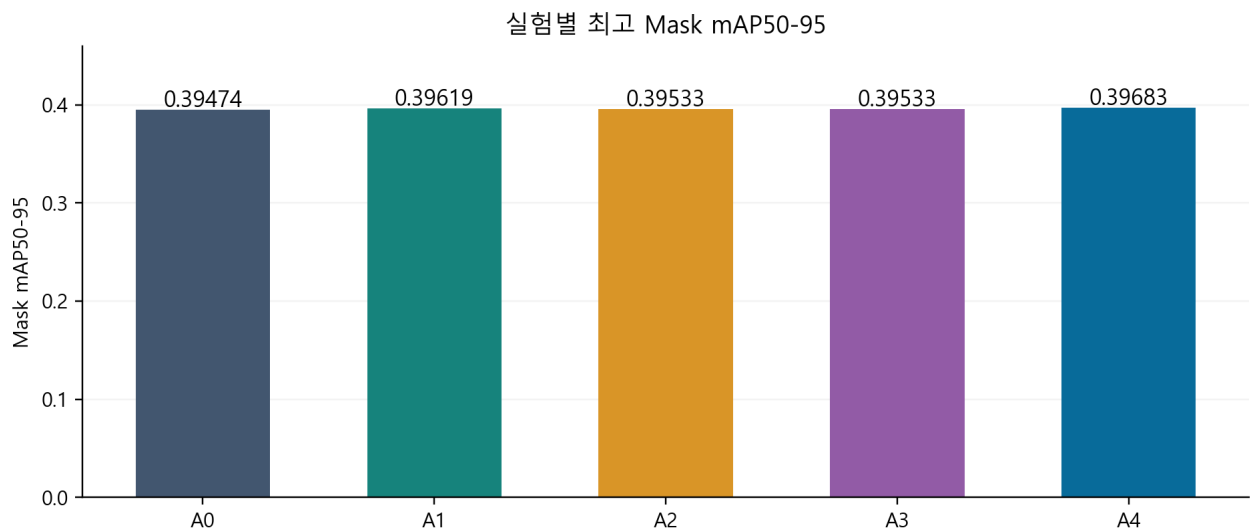


그림 4. 실험별 최고 Mask mAP50-95

저장된 학습 기록에서 A4 의 Mask mAP50-95 는 0.39683 으로 가장 높다. A0 의 0.39474 와 비교하면 0.00209, 즉 약 0.209%p 높다. A1 의 Mask Recall 은 0.56126 으로 다섯 실행 중 가장 높다.

비교 기준: 각 실행의 최고 Mask epoch 행. 수치는 원본 CSV 값을 소수점 다섯 자리로 표시하였다.

## 5.4 분할 성능 변화

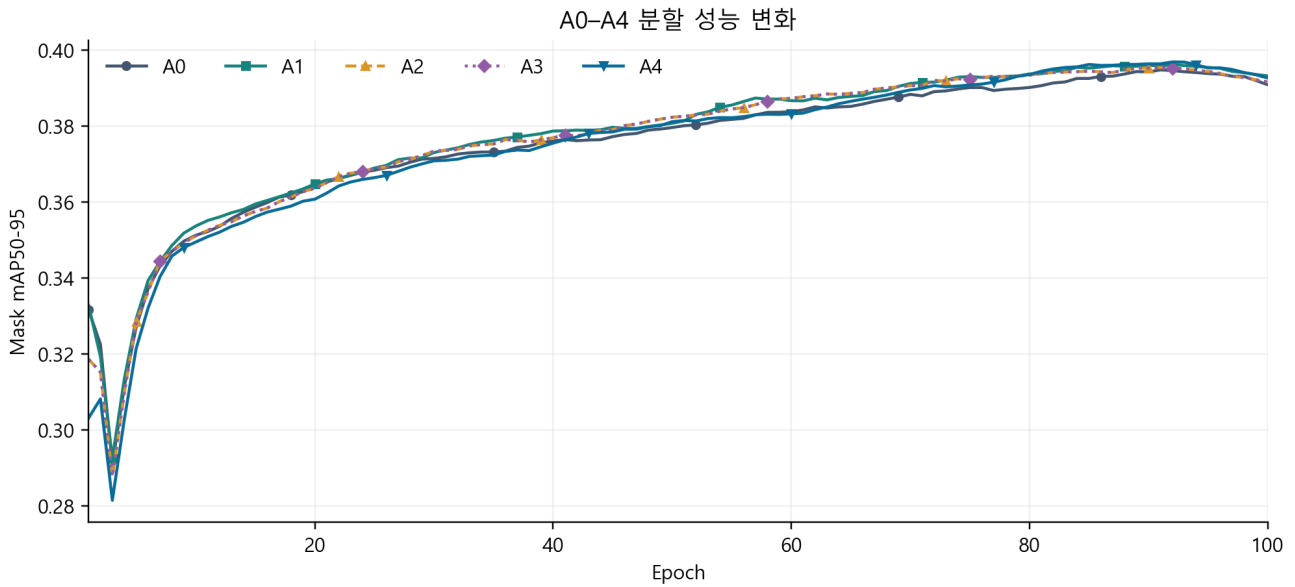


그림 5. A0~A4의 epoch 별 Mask mAP50-95

표 24. 최고 epoch 와 마지막 epoch 비교

| 실험 | 최고 Epoch | 최고 Mask mAP50-95 | 100 Epoch Mask mAP50-95 |
|----|----------|------------------|-------------------------|
| A0 | 91       | 0.39474          | 0.39085                 |
| A1 | 92       | 0.39619          | 0.39317                 |
| A2 | 91       | 0.39533          | 0.39155                 |
| A3 | 91       | 0.39533          | 0.39155                 |
| A4 | 92       | 0.39683          | 0.39261                 |

각 실험은 100 epoch의 기록을 갖고 있으며 최고 Mask 성능은 91~92 epoch에 나타난다. 마지막 epoch 수치와 최고 수치를 함께 제시하여 학습 종료 시점과 성능 선택 시점을 구분하였다.

A4는 최고 epoch 이후 학습 분할 손실이 감소하지만 검증 분할 손실은 증가하고 Mask mAP50-95는 낮아진다. 후속 실험에서는 단순 학습 연장보다 검증 성능을 기준으로 한 체크포인트 선정과 제한된 미세 조정을 검토할 수 있다.

그래프는 저장된 CSV를 사용해 다시 작성한 것이며, 보간·평활화 없이 epoch 별 값을 표시하였다.

## 5.5.1 A0 학습 결과

A0 는 YOLO26s-seg 기본 구조의 비교 기준이다.

A0 기본 모델 · 저장된 학습 기록

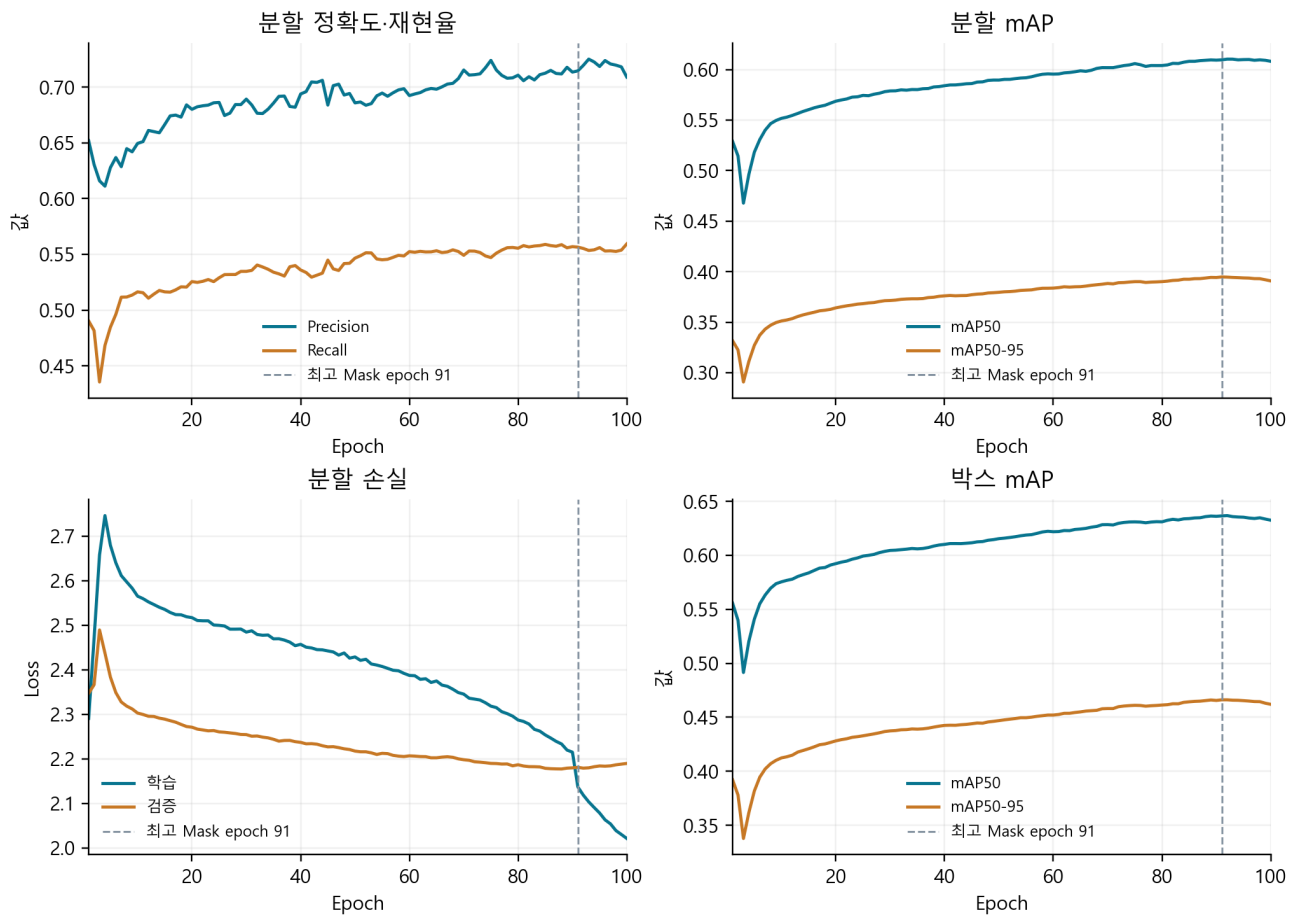


그림 6. A0 분할 정확도·mAP·손실·박스 성능 기록

표 25. A0 최고 Mask epoch의 성능

| 구분   | Precision | Recall  | mAP50   | mAP50-95 |
|------|-----------|---------|---------|----------|
| Box  | 0.71405   | 0.57778 | 0.63649 | 0.46617  |
| Mask | 0.71449   | 0.55642 | 0.60960 | 0.39474  |

저장된 학습 기록은 100 epoch이며, Mask mAP50-95 최고 시점은 91 epoch이다. 해당 값은 0.39474 이고, 100 epoch 값은 0.39085 이다.

자료: A0\_local/results.csv. 점선은 최고 Mask mAP50-95 epoch 를 나타낸다.

## 5.5.2 A1 학습 결과

A1 은 CARAFE 를 적용하여 업샘플링 단계의 특징 재조합을 변경한 구조이다.

A1 CARAFE · 저장된 학습 기록

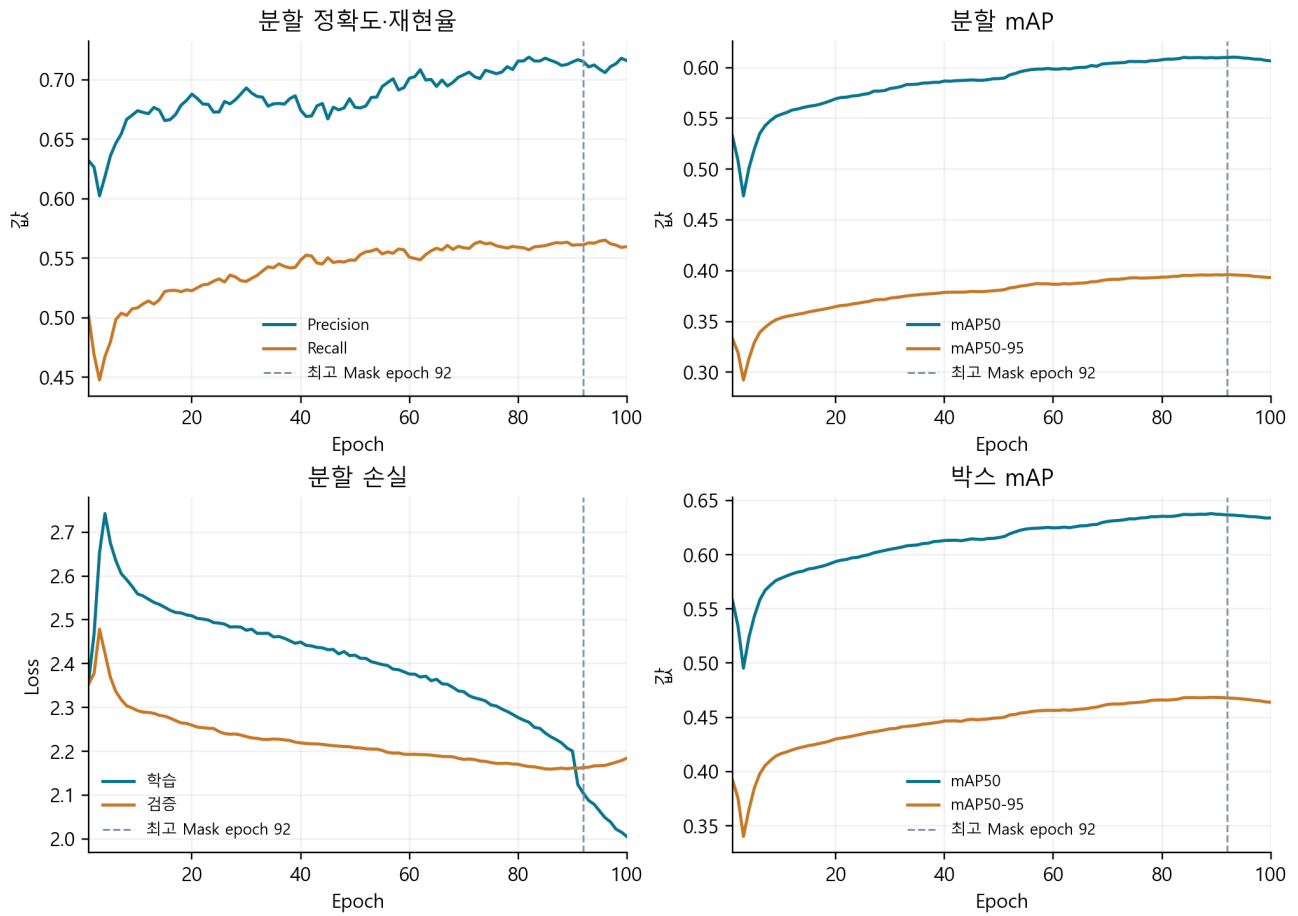


그림 7. A1 분할 정확도·mAP·손실·박스 성능 기록

표 26. A1 최고 Mask epoch 의 성능

| 구분   | Precision | Recall  | mAP50   | mAP50-95 |
|------|-----------|---------|---------|----------|
| Box  | 0.72561   | 0.57583 | 0.63665 | 0.46789  |
| Mask | 0.71518   | 0.56126 | 0.60992 | 0.39619  |

저장된 학습 기록은 100 epoch 이며, Mask mAP50-95 최고 시점은 92 epoch 이다. 해당 값은 0.39619 이고, 100 epoch 값은 0.39317 이다.

자료: A1\_local/results.csv. 점선은 최고 Mask mAP50-95 epoch 를 나타낸다.

### 5.5.3 A2 학습 결과

A2 는 ASPP 를 적용하여 고수준 특징에 다중 문맥을 결합하는 구조이다.

A2 ASPP · 저장된 학습 기록

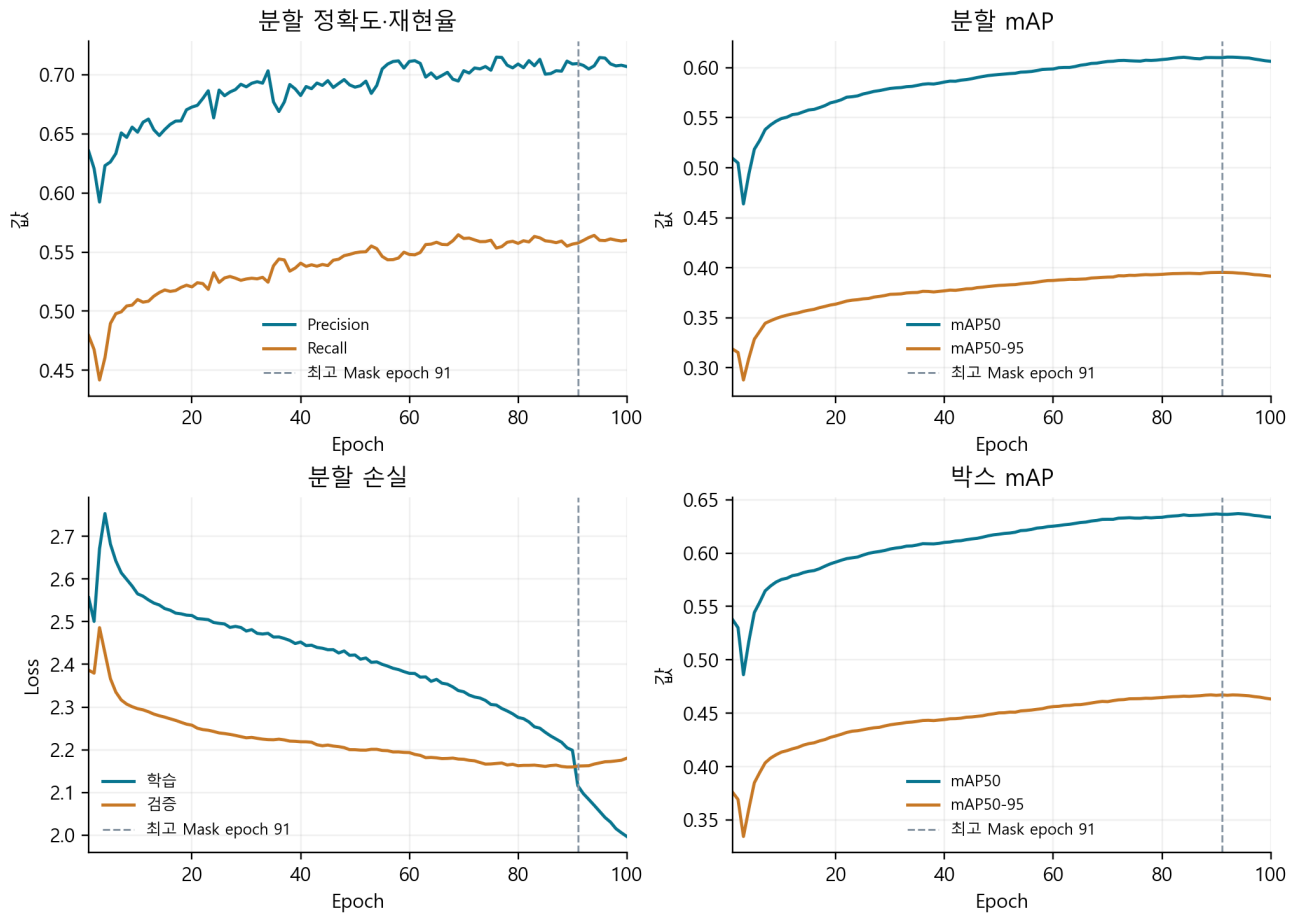


그림 8. A2 분할 정확도·mAP·손실·박스 성능 기록

표 27. A2 최고 Mask epoch 의 성능

| 구분   | Precision | Recall  | mAP50   | mAP50-95 |
|------|-----------|---------|---------|----------|
| Box  | 0.70833   | 0.57733 | 0.63635 | 0.46715  |
| Mask | 0.70948   | 0.55764 | 0.61014 | 0.39533  |

저장된 학습 기록은 100 epoch 이며, Mask mAP50-95 최고 시점은 91 epoch 이다. 해당 값은 0.39533 이고, 100 epoch 값은 0.39155 이다.

자료: A2\_local/results.csv. 점선은 최고 Mask mAP50-95 epoch 를 나타낸다.

### 5.5.4 A3 학습 결과

A3 는 CARAFE 와 ASPP 를 함께 적용한 구조이다.

A3 CARAFE + ASPP · 저장된 학습 기록

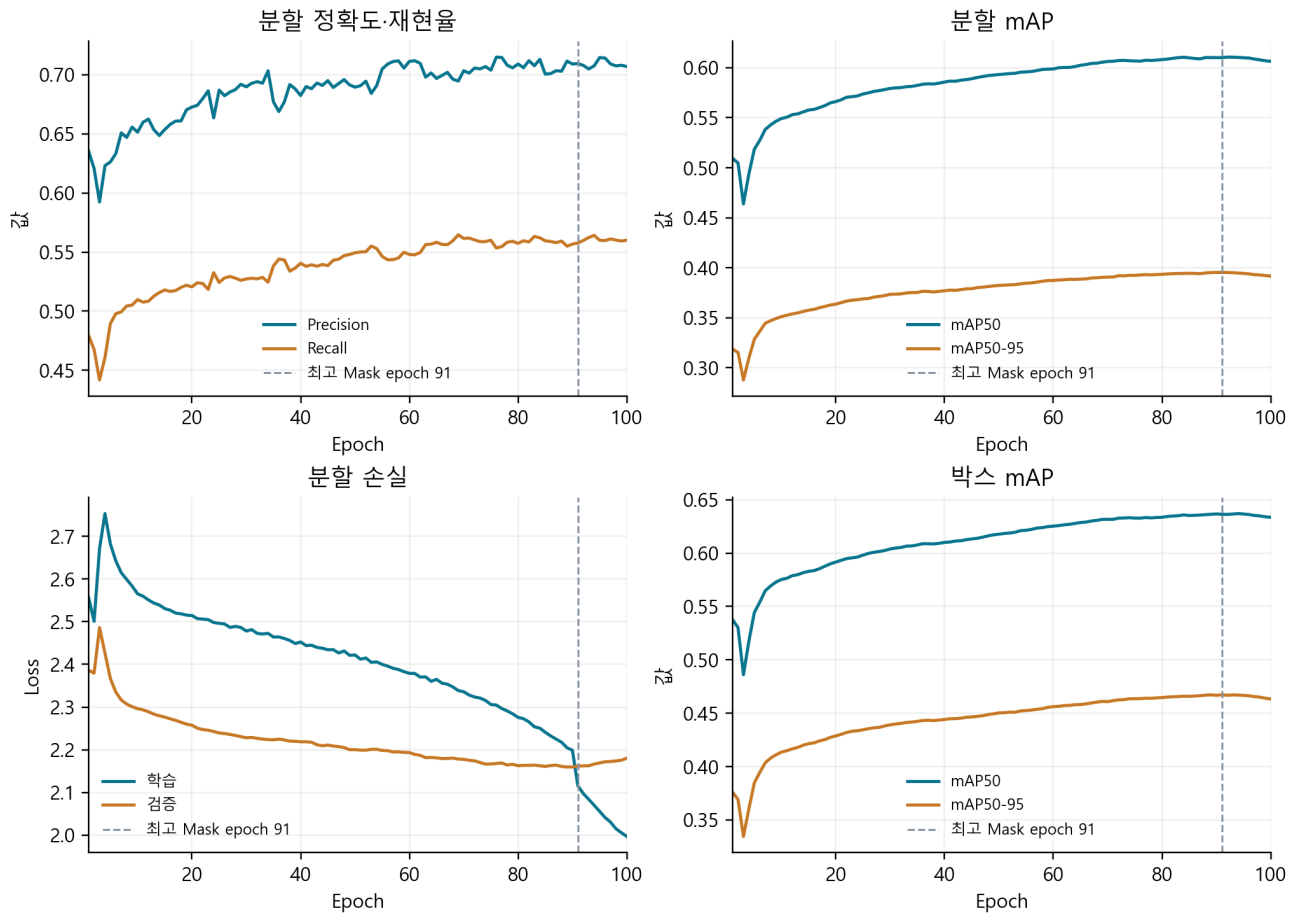


그림 9. A3 분할 정확도·mAP·손실·박스 성능 기록

표 28. A3 최고 Mask epoch 의 성능

| 구분   | Precision | Recall  | mAP50   | mAP50-95 |
|------|-----------|---------|---------|----------|
| Box  | 0.70833   | 0.57733 | 0.63635 | 0.46715  |
| Mask | 0.70948   | 0.55764 | 0.61014 | 0.39533  |

저장된 학습 기록은 100 epoch 이며, Mask mAP50-95 최고 시점은 91 epoch 이다. 해당 값은 0.39533 이고, 100 epoch 값은 0.39155 이다.

자료: A3\_local/results.csv. 점선은 최고 Mask mAP50-95 epoch 를 나타낸다.

### 5.5.5 A4 학습 결과

A4 는 CARAFE-ASPP 와 DeepLabV3+ 방식의 마스크 디코더를 결합한 구조이다.

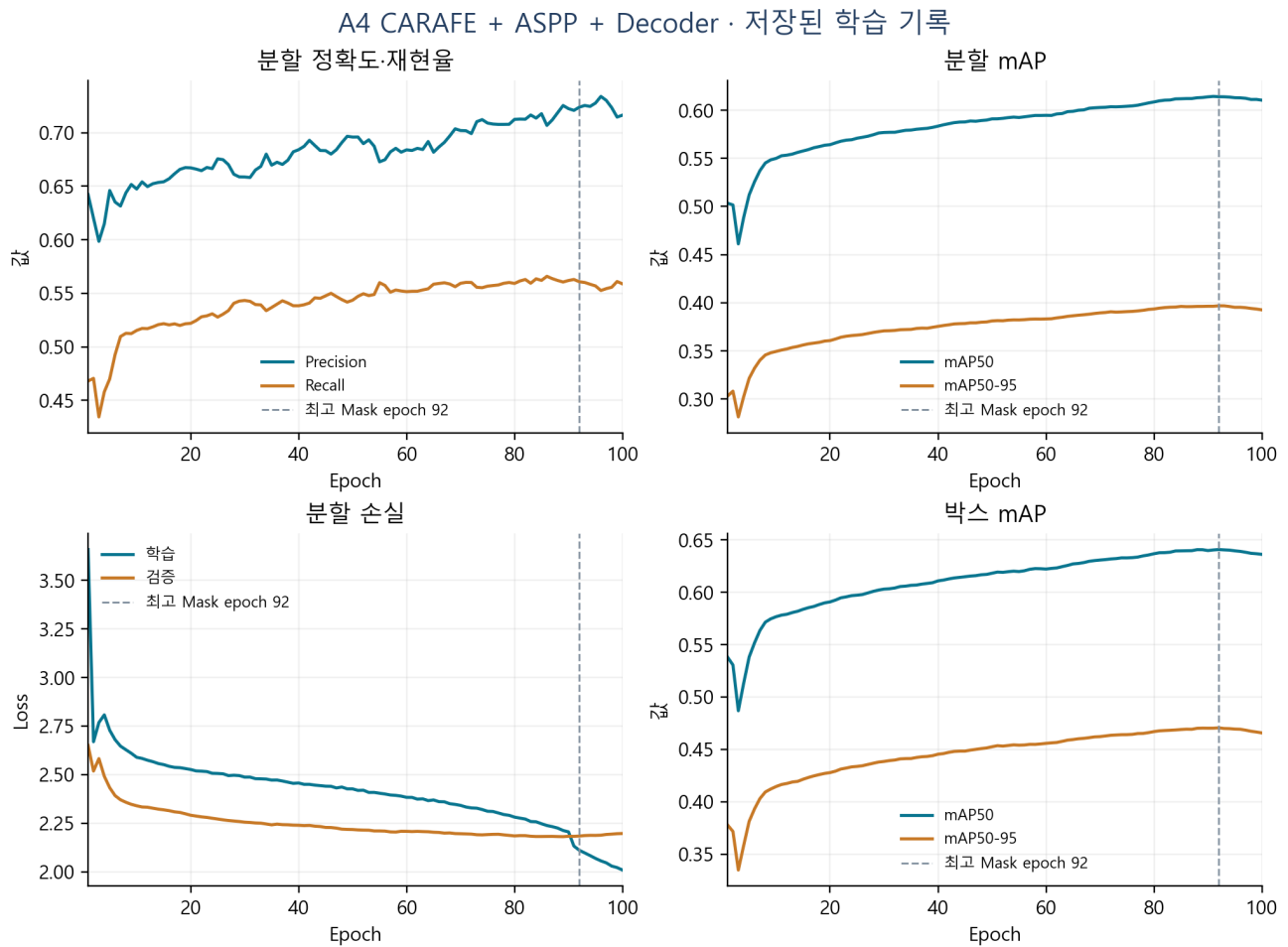


그림 10. A4 분할 정확도·mAP·손실·박스 성능 기록

표 29. A4 최고 Mask epoch 의 성능

| 구분   | Precision | Recall  | mAP50   | mAP50-95 |
|------|-----------|---------|---------|----------|
| Box  | 0.72519   | 0.58144 | 0.64082 | 0.47056  |
| Mask | 0.72381   | 0.56082 | 0.61404 | 0.39683  |

저장된 학습 기록은 100 epoch 이며, Mask mAP50-95 최고 시점은 92 epoch 이다. 해당 값은 0.39683 이고, 100 epoch 값은 0.39261 이다.

자료: A4\_local-2/results.csv. 점선은 최고 Mask mAP50-95 epoch 를 나타낸다.

## 5.6 공통 조건의 모델 평가

표 30. 공통 조건 평가 설정

| 항목                                | 설정                         |
|-----------------------------------|----------------------------|
| 비교 모델                             | A0, A4, 공식 yolo26s-seg.pt  |
| 검증 데이터                            | COCO val2017 / 5,000 이미지   |
| 입력 크기 / 배치                        | 640 / 4                    |
| 장비 / 정밀도                          | Quadro P2200, GPU 0 / FP32 |
| Confidence / IoU / max detections | 0.001 / 0.7 / 300          |
| NMS 설정                            | 외부 NMS, nms=None           |

표 31. 공통 조건의 전체 Mask 성능

| 모델    | Precision | Recall | mAP50 | mAP50-95 |
|-------|-----------|--------|-------|----------|
| A0    | 0.717     | 0.557  | 0.609 | 0.389    |
| A4    | 0.725     | 0.558  | 0.612 | 0.392    |
| 공식 모델 | 0.716     | 0.564  | 0.618 | 0.393    |

표 32. 공통 조건의 전체 Box 성능

| 모델    | Precision | Recall | mAP50 | mAP50-95 |
|-------|-----------|--------|-------|----------|
| A0    | 0.717     | 0.577  | 0.637 | 0.467    |
| A4    | 0.728     | 0.580  | 0.640 | 0.470    |
| 공식 모델 | 0.720     | 0.590  | 0.649 | 0.476    |

기존 평가 자료의 원정밀도 값 기준으로 A4 의 전체 Mask AP50-95 는 0.39161 이며 A0 대비 0.00216 높다. 공식 체크포인트의 0.39285 보다는 0.00124 낮다. 따라서 A4 가 기본 학습 모델보다 개선된 부분과 공식 모델 대비 남은 차이를 함께 확인할 수 있다.

출처: 기존 yolo26\_seg\_model\_comparison.md 및 A4\_class\_finetuning\_plan.md. 표는 기존 공통 조건 결과를 소수점 세 자리로 표시하였다.

## 5.7 교통 객체 클래스별 성능

표 33. 주요 클래스의 Mask AP50-95

| 클래스        | 정답 객체 수 | A0    | A4    | 공식 모델 | A4 - A0  |
|------------|---------|-------|-------|-------|----------|
| person     | 10,777  | 0.466 | 0.465 | 0.468 | -0.00137 |
| bicycle    | 314     | 0.201 | 0.196 | 0.212 | -0.00539 |
| car        | 1,918   | 0.375 | 0.379 | 0.382 | +0.00383 |
| motorcycle | 367     | 0.368 | 0.378 | 0.388 | +0.00910 |
| bus        | 283     | 0.650 | 0.649 | 0.654 | -0.00095 |
| truck      | 414     | 0.348 | 0.356 | 0.356 | +0.00776 |

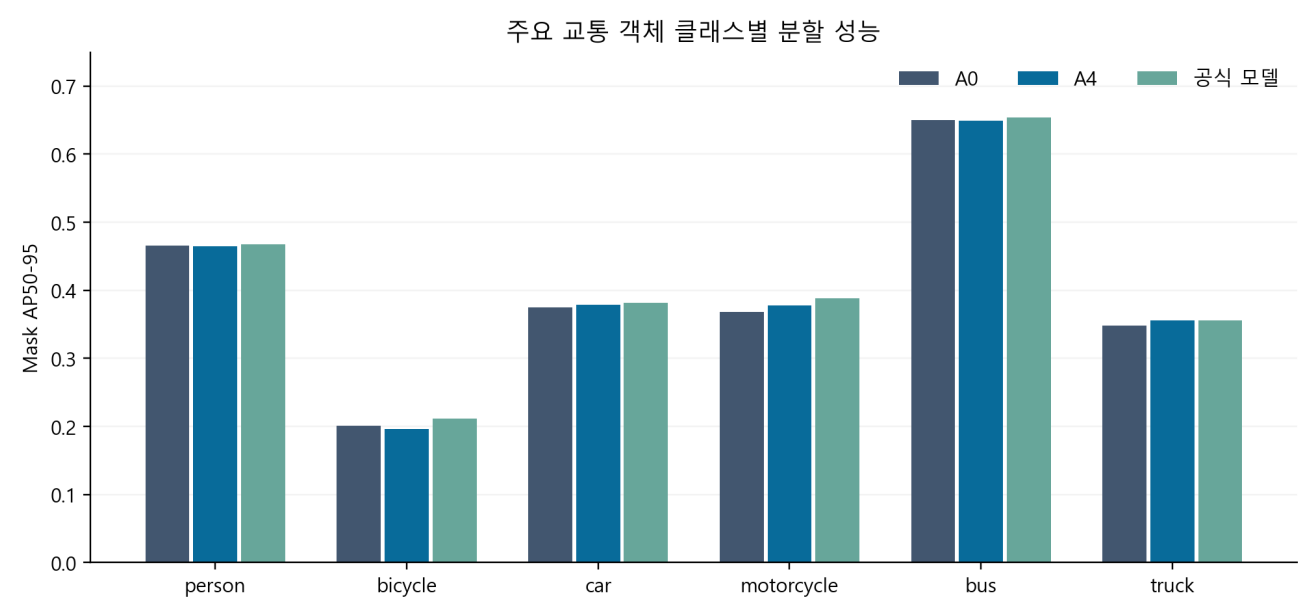


그림 11. 주요 교통 객체의 클래스별 분할 성능

A4 는 A0 대비 car·motorcycle·truck 의 Mask AP50-95 가 높고, person·bicycle·bus 는 낮다. 특히 motorcycle 의 증가 폭은 약 0.00910 이다. truck 은 공식 모델과 표의 반올림 값이 같으며, 원정밀도에서는 A4 0.35610 과 공식 모델 0.35596 으로 차이가 작다.

표의 AP 는 소수점 세 자리, 차이는 기존 평가 자료의 원정밀도 기준이다. 정답 객체 수와 클래스별 지표는 기존 모델 비교 문서를 사용하였다.

## 5.8 UniDepth 성능 평가 작성란

거리 측정 평가 조건과 결과를 추가할 수 있도록 아래 표와 그래프 영역을 마련하였다.

표 34. UniDepth 평가 조건·성능

| 항목          | 설정 / 측정값 | 비고 |
|-------------|----------|----|
| 평가 데이터셋     |          |    |
| 정답 거리 취득 방식 |          |    |
| 평가 거리 범위    |          |    |
| 평가 장비 / 해상도 |          |    |
| 거리 오차 지표    |          |    |
| 정확도 지표      |          |    |
| 추론 시간       |          |    |
| 장면별 결과      |          |    |

거리 측정 결과·오차 그래프 작성 공간

## 5.9 시스템 검증 항목

표 35. 통합 기능 검증 시나리오

| 영역    | 확인할 시나리오                 | 현재 근거            |
|-------|--------------------------|------------------|
| 인증·권한 | 조회자 쓰기 요청 차단, 역할별 차량 API | 라우터·미들웨어·테스트 코드  |
| 차량 위치 | BIMS/재생 전환, 지연 표시, 기록 보정 | 추적·지도·소스 설정 코드   |
| 경로 계산 | 높이·중량·너비·회전 제한, 경로 없음    | 그래프·경로 코드와 테스트   |
| 영상·센서 | 원본 시각 매칭, 탐지·GPS 동기화     | 타임라인·센서 테스트 코드   |
| 녹화·재생 | 세그먼트 등록, PTS 조회, URL 발급  | 녹화 API·재생 테스트 코드 |
| 가상 배차 | 중복 수락, 경유지 진행, 자동 추종 전환  | 배차·경유지·점유 테스트 코드 |
| 도로 통제 | 점유 도로 차단 거부, 대체 경로·정지    | 가상 통제 구현과 테스트    |

저장소에는 위 동작을 확인하기 위한 테스트 코드가 포함되어 있다. 본 보고서 작성에서는 테스트 전체 실행이나 운영 장비의 통합 시연을 수행하지 않았으므로, 코드 확인을 테스트 통과 결과로 기재하지 않는다.

표 36. 실행 환경별 성능 측정 작성란

| 항목           | 장비 / 조건 | 측정값 |
|--------------|---------|-----|
| 영상 송출 FPS    |         |     |
| 분할 추론 시간     |         |     |
| 통합 추론·표시 지연  |         |     |
| 경로 계산 시간     |         |     |
| 녹화·재생 동기화 오차 |         |     |

## 6. 프로젝트 결과

### 6.1 구현 결과

표 37. 기능별 구현 구성

| 영역     | 구현 내용                          | 관련 구성                |
|--------|--------------------------------|----------------------|
| 업무 API | 인증, 차량 CRUD, 운행 생성·취소·단말 시작·종료 | Node 모듈·PostgreSQL   |
| 통합 지도  | BIMS·단말 GPS, 계획 경로, 지연·소스 표시   | Routing·관제 웹         |
| 경로 탐색  | OSM 그래프와 제한 데이터, 차량 제약 A*      | Routing/Tracking     |
| 영상 분석  | WebRTC·추론 feed, 객체 분할·추적·거리 상태 | Android·Relay·Vision |
| 운행 기록  | 세그먼트·탐지 저장, 재생 URL·타임라인        | MinIO·Node·Vision·웹  |
| 가상 배차  | 경로 미리보기, 요청 수락, 서버 시뮬레이션       | Node·Routing·웹       |
| 도로 통제  | 정체·차단, 점유 검사, 차량별 재경로          | 가상 상태·그래프 오버레이       |
| 부가 도우미 | 차량 스냅샷과 문서 검색 기반 답변            | Node·LLM 서비스         |

### 6.2 분할 모델 개발 결과

YOLO26s-seg 를 기준으로 A0~A4 구조를 구성하고 학습 기록과 평가 그래프를 확보하였다. A4 는  
고수준 문맥과 저수준 상세 특징을 결합하는 분할 경로를 포함한다. 저장된 학습 검증 기록에서 A4 는  
전체 Mask mAP50-95 0.39683 을 기록하였다.

공통 조건 평가에서는 기본 학습 모델 대비 개선이 확인되었으며, 클래스별 결과를 통해 성능이 개선된  
객체와 후속 보완이 필요한 객체를 구분하였다. UniDepth 의 상세 구성과 평가 결과는 본 문서의 빈 표에  
추가하도록 마련하였다.

## 6.3 주요 기능 시연 — 관제·영상·기록

### 6.3.1 차량 관제와 실시간 영상

1. 운영자가 로그인하고 지도에서 차량을 선택한다.
2. 선택 차량의 현재 위치·상태·계획 경로를 확인한다.
3. Android 가 배정 운행을 시작하고 WebRTC 로 카메라와 센서를 보낸다.
4. Live View 에서 객체 클래스·박스·마스크·거리 표시를 확인한다.
5. 클래스별 표시 방식을 조절하고 해당 프레임의 GPS 와 지도 위치를 확인한다.

### 6.3.2 GPS 지연과 소스 전환

1. 실시간 BIMS 위치에서 마지막 관측 시각과 지연 상태를 확인한다.
2. 기록 경로 보정 설정을 바꾸어 선택적 보정 동작을 확인한다.
3. BIMS 실시간과 재생 소스를 전환하고 지도·차량 상태 갱신을 확인한다.

### 6.3.3 운행 기록 재생

1. 운행의 녹화 목록을 열고 재생할 영상 구간을 선택한다.
2. 발급된 접근 URL 로 영상을 불러온다.
3. 영상 PTS 를 기준으로 탐지 결과와 GPS 를 표시한다.
4. 타임라인을 이동하고 영상·탐지·지도 상태가 같은 구간을 가리키는지 확인한다.

시연 절차는 구현된 사용자 흐름을 정리한 것이다. 운영 장비에서 실행한 시연 결과는 별도로 기록한다.

## 6.4 주요 기능 시연 — 가상 배차·도로 통제

### 6.4.1 가상 배차

1. 시나리오와 사용 가능한 가상 차량을 선택한다.
2. 출발지·목적지·경유지를 입력하고 경로를 미리 확인한다.
3. 모의 운전자에게 배차를 요청한다.
4. 수동 또는 설정된 자동 수락 후 차량 이동을 확인한다.
5. 가상 운행 상태와 경로 버전, 운영자 이벤트 기록을 확인한다.

### 6.4.2 정체·차단 대응

1. 지도에서 정체 구역을 만들고 영향 도로를 확인한다.
2. 자동 추종을 켜 차량이 새 경로를 계산하는 흐름을 확인한다.
3. 차단 구역을 설정하고 자동 추종을 끈 차량의 대기 동작을 확인한다.
4. 자동 추종 OFF→ON 전환 후 현재 위치에서 남은 경로를 다시 계산한다.
5. 가상 차량이 점유한 도로를 차단하는 요청이 거부되는지 확인한다.

### 6.4.3 경유지 변경

운행 중 경유지를 추가·변경하고 현재 위치에서 목적지까지의 남은 경로를 확인한다. 실제 차량·녹화 기능과 가상 주행이 분리되어 동작하는지 함께 검토한다.

## 6.5 프로젝트 성과 및 한계점

### 6.5.1 프로젝트 성과

- 업무·경로 계산·미디어·비전 서비스를 하나의 제품 흐름으로 연결하였다.
- 차량 출처와 위치 출처를 구분하고 실제·가상 운행 데이터를 분리하였다.
- 녹화 영상과 탐지·GPS 정보를 동일한 원본 시간축으로 조회하는 구조를 마련하였다.
- 가상 배차와 통제 상황을 서버 상태로 관리하여 관제 시나리오를 구성하였다.
- 분할 모델의 구조별 학습 기록과 교통 객체별 평가 자료를 정리하였다.

### 6.5.2 한계점

표 38. 주요 한계와 보완 항목

| 항목      | 현재 한계                          | 보완 방향               |
|---------|--------------------------------|---------------------|
| 실환경 일반화 | COCO 평가만으로 부산 주행 성능을 확정할 수 없음  | 주행·야간·가림·원거리 자료 확보  |
| 객체별 성능  | A4 의 person·bicycle 등 성능 보완 필요 | 목표 클래스 미세 조정과 회귀 검증 |
| 거리 측정   | 상세 모델·정답 거리 평가 자료 미가입          | 모델 자료와 거리 오차 결과 추가  |
| 영상 확장   | 단일 Android 스트림 관제 구성           | 필요 시 스트림·세션 라우팅 확장  |
| 도로 제약   | 시연용 차량 프로파일·지역 데이터             | 현장 제한값 검증·데이터 갱신    |
| 운영 검증   | 본 문서 작성에서 GPU·단말 E2E 미실행       | 원래 장비에서 통합·장시간 검증   |

## 7. 결론 및 향후 계획

### 7.1 프로젝트 요약

본 프로젝트는 차량 위치와 경로, 카메라 영상의 AI 분석과 운행 기록을 연결하는 통합 관제 시스템을 구성하였다. Node 는 업무 데이터를 관리하고, Routing 은 계산과 위치 추적을 담당하며, Go 릴레이와 Vision 은 영상·추론 경로를 담당한다. 가상 배차·도로 통제는 실제 운행과 분리된 시뮬레이션으로 제공한다.

### 7.2 결론

Custom YOLO26-seg 는 CARAFE·ASPP·디코더의 구조별 실험을 통해 분할 성능을 비교할 수 있는 자료를 확보하였다. A4 의 성능은 기본 학습 모델보다 높아진 항목이 있지만, 전체 및 클래스별 결과를 함께 보아야 한다. UniDepth 는 거리 측정을 담당하며 모델 상세와 평가 자료를 보완할 공간을 마련하였다.

### 7.3 향후 개선 방향

표 39. 후속 개선 계획

| 영역    | 개선 내용                               | 확인 기준                |
|-------|-------------------------------------|----------------------|
| 분할 모델 | 실제 주행 데이터, person·bicycle 보완, 미세 조정 | 목표 클래스 AP 와 전체 성능 회귀 |
| 거리 모델 | 상세 모델 자료·카메라 보정·정답 거리 정리            | 거리대·장면별 오차           |
| 영상·기록 | 장시간 송출·재연결·녹화 안정성                   | 프레임 지연·누락·동기화 오차     |
| 경로·통제 | 제한 데이터 검증과 계산 최적화                   | 통행 제한 반영·경로 응답 시간    |
| 업무 확장 | 경로 이탈·알림 등 스키마 기능 연결                | 실제 이벤트·권한·저장 검증      |

### 7.4 향후 활용 방안

차량 운영 교육과 관제 시연, 도로 통제 시나리오 검토에 활용할 수 있다. 실제 운행 적용을 위해서는 도로 제한과 거리 정확도, 운영 장비의 송출·녹화·재생 안정성을 추가로 검증해야 한다.

## 작성 근거 및 자료 위치

표 40. 본문 작성에 사용한 자료

| 자료                                                     | 활용 범위                             |
|--------------------------------------------------------|-----------------------------------|
| P4 통합 백엔드 시스템 설계 문서.md                                 | 요구사항·유스케이스·서비스 책임·DB 설계           |
| 현재 저장소의 서비스 코드·Compose·Prisma 스키마                      | 실제 구현·인터페이스·설정 확인                 |
| yolo_carafe_aspp / experiments/configs 및 custom_seg.py | 분할 모델 구조·모듈 설정                    |
| yolo_carafe_aspp / runs/segment/experiments/results    | A0~A4 args.yaml·results.csv·학습 자료 |
| experiments/notes/yolo26_seg_model_comparison.md       | 공통 조건 전체·클래스별 평가 결과               |
| experiments/notes/A4_class_finetuning_plan.md          | 원정밀도 클래스별 차이·후속 실험 방향             |
| 이전 프로젝트 기획서·보고서 PDF                                    | 목차·서식·문체 참고                       |

## 자료 수정 안내

본문과 표는 Word 에서 수정할 수 있다. 학습 그래프는 source/data 의 실행별 CSV 에서 생성되며, 그림 원본은 source/assets 에 있다. 수치를 변경할 때 성능 표·본문 설명·그래프를 함께 갱신한다. UniDepth 표와 그림 영역은 해당 자료를 추가하여 완성한다.

## 검증 범위

프로젝트 설명은 소스·설정·설계 자료를 확인한 결과이며, 모델 수치는 기존 기록을 정리한 것이다. 문서 작성 과정에서 새 학습, GPU 추론, 실차 송출 또는 운영 배포를 실행하지 않았다.